# NYC TAXI

Presented by : Group 8

# Table Of Content


Problem Statement

DataOps

MLOps

DevOps

Conclusion

# 1. Problem

How can a **private ride-hailing company** predict **zone-level ride demand** in advance so that drivers can be proactively repositioned to areas with higher expected demand?

# Data

Taxi & Limousine Commission

## TLC Trip Record Data

20-40 million individual trip data every month

## BIG Data

# Taxi

## Classic Yellow Taxi

3.56 million individual trip data every month

## High-Volume For-Hire Services

19.8 million individual trip data every month

# BIG Data

# Data

Taxi & Limousine Commission

## TLC Trip Record Data

20-40 million individual trip data every month

## Open-Meteo

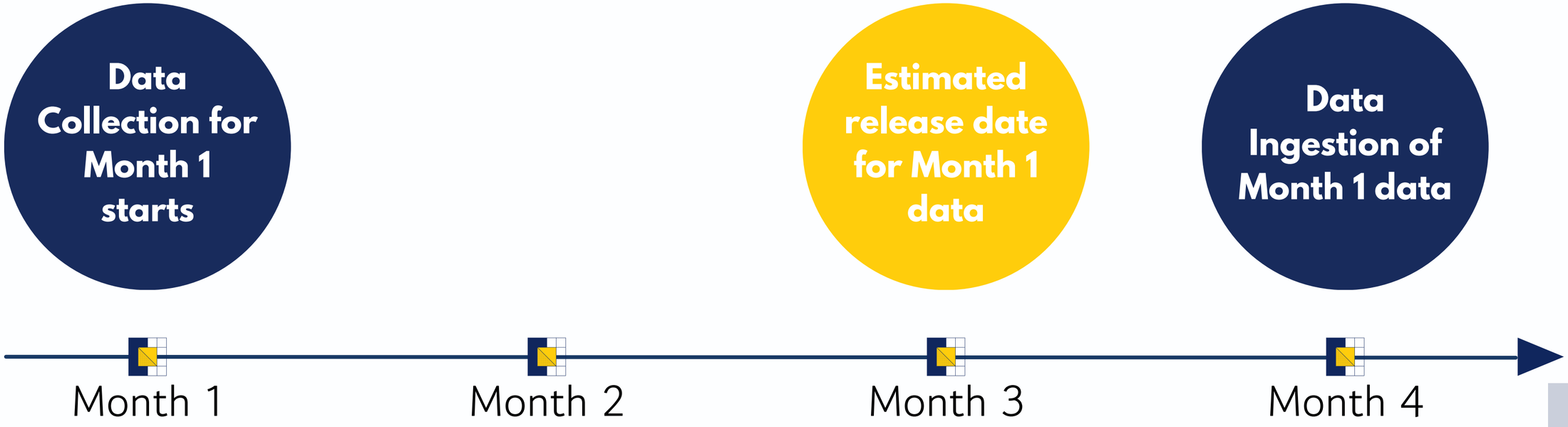
Daily weather data

# BIG Data

# DATAOPS

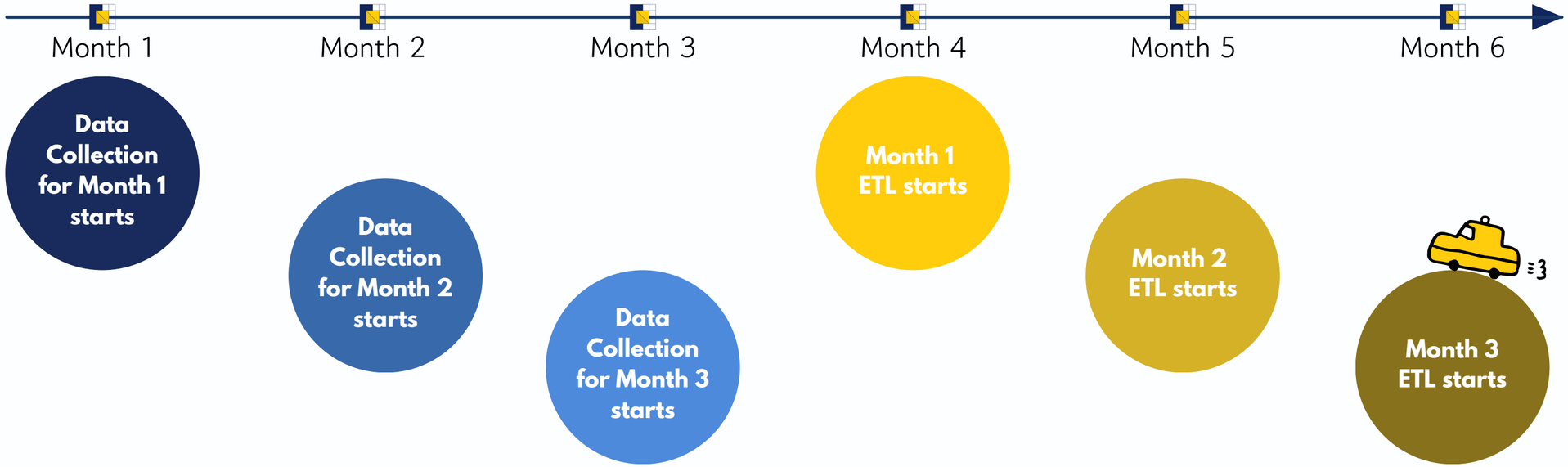
# 2. DataOps

INGESTION - 3 MONTH LAGS



# 2. DataOps

## MONTHLY DAGS



# 2. DataOps

## INGESTION - WEATHER

Daily weather data was ingested per borough

- precipitation
- temperature
- wind speed

# 2. DataOps

TRANSFORMATION

1. Combine Yellow Taxi and HVFHS Taxi Data
2. Drop columns unrelated to demand
    a. eg. some columns informed how the data was collected
3. Aggregate to reduce granularity
    a. Imposes severe computational and storage overhead
    b. Not interested with the individual trip itself, but the volume of demand within a given pickup zone over a defined time interval

# 2. DataOps

## TRANSFORMATION

843,752,700 rows of individual trips

| pulocationid | pickup datetime | dropoff datetime | ... |
|---|---|---|---|
| | | | |
| | | | |

aggregate on
pulocationid & hourts →

6,625,674 rows of zone-demand data

| pulocation id | hour_ts | ... | target_ demand |
|---|---|---|---|
| | | | |
| | | | |

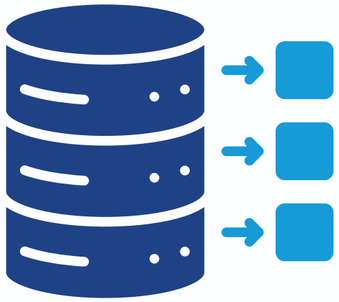
* hourts is the pickupdatetime rounded down to the nearest hour

# 2. DataOps

LOADING

**Parquet**

memory-safe chunks
of 5,000 rows



**PostgreSQL**

# Flagging Data Anomalies

**Execution Monitoring: Custom @monitor decorators and SMTP callbacks.**

**Automated Data Quality:**

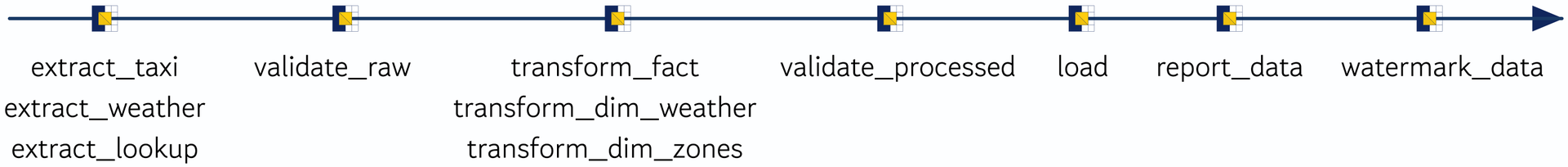
- On failure: triggers an immediate email alert (embeds the exact DAG name, run ID, a direct link to the logs) as an HTML table.
- On success: report_data task automatically dispatches a data quality email report (attaches distribution charts and null rate summaries) to the stakeholders.

# Data Lineage

Airflow + OpenLineage + Marquez UI

- Every job, run, and dataset is tracked and stored in a dedicated Marquez PostgreSQL database
- Instant root-cause debugging and total system transparency

# 2. DataOps


extract_taxi
extract_weather
extract_lookup
validate_raw
transform_fact
transform_dim_weather
transform_dim_zones
validate_processed
load
report_data
watermark_data

# MLOPS

# Pipeline

**EDA**

**Feature Engineering**

**Model Training + Experimentation on MLflow**

6.4M rows, 3 years of data
Built with PySpark for scalability

# EDA

- Sampled ~1M rows
- Demand varies by time & location
- Strong temporal patterns observed

# Feature Engineering

## Temporal Features

hour
day_of_week
month
is_weekend
hour_sin
hour_cos
dow_sin
dow_cos

# Feature Engineering

## Temporal Features

hour
day_of_week
month
is_weekend
hour_sin
hour_cos
dow_sin
dow_cos

## Weather Features

temperature_mean
precipitation_sum
is_rainy
is_heavy_rain
is_hot

# Feature Engineering

## Temporal Features

hour
day_of_week
month
is_weekend
hour_sin
hour_cos
dow_sin
dow_cos

## Weather Features

temperature_mean
precipitation_sum
is_rainy
is_heavy_rain
is_hot

## Lag Features

demand_lag_1
demand_lag_2
demand_lag_24
rolling_mean_3h

# Feature Engineering

## Temporal Features

hour
day_of_week
month
is_weekend
hour_sin
hour_cos
dow_sin
dow_cos

## Weather Features

temperature_mean
precipitation_sum
is_rainy
is_heavy_rain
is_hot

## Lag Features

demand_lag_1
demand_lag_2
demand_lag_24
rolling_mean_3h

## Encoding Features

Borough (one-hot)
Service Zone (one-hot)

# Feature Engineering

## Temporal Features

hour
day_of_week
month
is_weekend
hour_sin
hour_cos
dow_sin
dow_cos

## Weather Features

temperature_mean
precipitation_sum
is_rainy
is_heavy_rain
is_hot

## Lag Features

demand_lag_1
demand_lag_2
demand_lag_24
rolling_mean_3h

## Encoding Features

Borough (one-hot)
Service Zone (one-hot)

**Stored in Postgres Feature Store**

# Training & MLflow

- Time-based split (70/15/15)
- Models: Linear Regression, Random Forest, Gradient Boosted Trees
- Metrics: RMSE, MAE, SMAPE

Table Chart Evaluation Experimental

| Run Name | Created | Dataset | Duration | Source | Models |
|---|---|---|---|---|---|
| sassy-grub-770 | 2 minutes ago | - | 1.0s | train.py | - |
| gbt | 15 minutes ago | - | 13.5min | train.py | spark |
| random_forest | 27 minutes ago | - | 12.1min | train.py | spark |
| linear_regression | 32 minutes ago | - | 4.9min | train.py | spark |

# Results & Model Selection

| Model | Val RMSE | Val SMAPE |
|---|---|---|
| Linear Regression | 36.5 | 0.29 |
| Random Forest | 37.0 | 0.23 |
| Gradient Boosted Trees | 37.3 | 0.24 |

# Results & Model Selection

| Model | Val RMSE | Val SMAPE |
|---|---|---|
| Linear Regression | 36.5 | 0.29 |
| Random Forest | 37.0 | 0.23 |
| Gradient Boosted Trees | 37.3 | 0.24 |

**SELECTION STRATEGY**

Top 2 models by Val RMSE

If RMSE difference ≤ 1.0 → select by SMAPE

Else → select by lowest RMSE

# Results & Model Selection

| Model | Val RMSE | Val SMAPE |
|---|---|---|
| Linear Regression | 36.5 | 0.29 |
| Random Forest | 37.0 | 0.23 |
| Gradient Boosted Trees | 37.3 | 0.24 |

**SELECTION STRATEGY**

Top 2 models by Val RMSE

If RMSE difference ≤ 1.0 → select by SMAPE

Else → select by lowest RMSE

# Results & Model Selection

| Model | Val RMSE | Val SMAPE |
|---|---|---|
| Linear Regression | 36.5 < 1.0 | 0.29 |
| Random Forest | 37.0 | 0.23 |
| Gradient Boosted Trees | 37.3 | 0.24 |

**SELECTION STRATEGY**

Top 2 models by Val RMSE

If RMSE difference ≤ 1.0 → select by SMAPE

Else → select by lowest RMSE

# Results & Model Selection

| Model | Val RMSE | Val SMAPE |
|---|---|---|
| Linear Regression | 36.5 | 0.29 |
| Random Forest | 37.0 | 0.23 |
| Gradient Boosted Trees | 37.3 | 0.24 |

**SELECTION STRATEGY**

Top 2 models by Val RMSE

If RMSE difference ≤ 1.0 → select by SMAPE

Else → select by lowest RMSE

# Three-Layer Drift Detection

- Feature Drift: relative mean shift for numeric features and proportional shift for one-hot categorical features
- Label Drift: monitors demand (target variable) distribution using mean, std and $50^{th}$ and $90^{th}$ percentiles
- Model Drift: runs live production model against live data and computes RMSE degradation ratio

# Smart Alerting Threshold

- Retraining is triggered only if:
  - Critical feature drift detected
  - Multiple high-drift features
  - Significant label shift
  - Model performance degrades

# Two DAGs Architecture

- Drift monitoring:
  - Scheduled daily
  - Detect drift, save reports, trigger retraining only when needed
- Training:
  - Scheduled monthly or drift-triggered retraining
  - Feature engineering, model training and evaluation
  - Context aware training pipeline

# DEVOPS

# Q&A

# THANK YOU

Presented by : Group 8